WORKSHOP 研讨/分享/感悟

仓颉社区中的灵感碰撞 / 社区先行者的干货分享

仓颉 Cangjie

编程语言的High-Level IR设计 --以仓颉为例

主讲人：涂功



目录

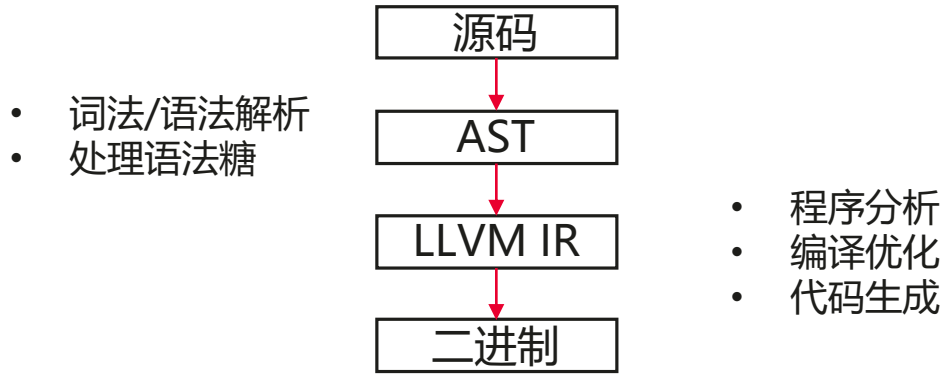
- 背景和动机
- 业界优秀实践简介
- Cangjie High-Level IR设计

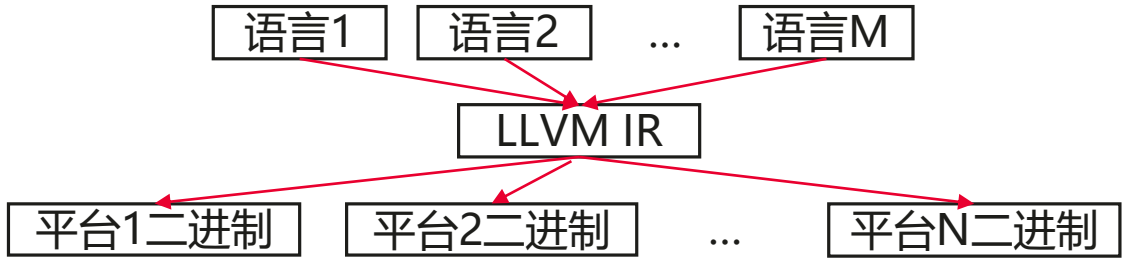
目录

- 背景和动机
- 业界优秀实践简介
- Cangjie High-Level IR设计

传统编译架构通常包含AST和IR两种不同抽象层次的中间表达

- 以目前业界主流的LLVM编译架构为例：





- LLVM IR作为连接前端和后端的中枢，很好的解决了 M种语言×N种平台 的编译器实现分裂问题

- 围绕LLVM IR，社区积累非常多技术资产（大量分析优化算法、不同目标平台的代码生成实现等等），新兴的编程语言通常也希望对接到LLVM IR上来复用上述技术资产

现代语言设计需求与传统编译架构之间存在Gap

Gap 1：更强的编译期代码检查

源代码（CJ）：

```
// helloWorld.cj
func foo(cond: Bool) {
    let idx = 100
    if (cond) {
        idx = 100
    }
    let arr = [1, 2, 3, 4]
    return arr[idx]
}
```

要点：
- 分析出idx=100
- 理解arr[100]是一个数组元素访问操作

预期编译结果：

```
 ==> helloWorld.cj:7:12:
  |
3 |     return arr[idx]
  |            ^^^^^^^^ array
index idx is past the end of array
(which contains 4 elements)
  |
```

- AST是一个树形的层级结构，非常贴近源码，**难以进行数据流分析**



- LLVM IR具备数据流分析能力，但**无法感知高层语义**

arr[idx]在AST上已解糖为一个函数调用，
LLVM IR上无法感知函数的语义行为

```
%12 = call i64 @"_ZN8std$core5ArrayIl2[]El"(i8* null,
%"record._ZN8std$core5ArrayIlE"* %arr, i64 1)
```

仓颉 Cangjie

现代语言设计需求与传统编译架构之间存在Gap

Gap 2：更高抽象层次的语义信息带来的编译优化机会

源代码（CJ）及预期效果：

```
// 注意仓颉中非public的class是不对外可见的
open class Base {
    open func foo() { 0 }
}

open class Sub1 <: Base {
    open func foo() { 1 }
}

func test(Sub1 obj) {
    // 此处的虚函数调用可优化为Sub1::foo的直接调用
    obj.foo()
}

func test(arr: Array<Int64>) {
    map(
}
```

要点：
- 分析出obj的实际类型是Sub1
- 理解Base和Sub包外不可见
- 理解Base和Sub1的虚表内容

- AST中不存在虚表，相关语义信息隐含在代码中

- LLVM IR没有原生的虚表概念，通常有两种思路：

> 一、将虚表具体实现为LLVM IR上支持的数据结构（如数组），虚函数的调用具体实现为了数据结构中的元素访问和函数指针调用。此时**虚函数调用优化问题转化为了常量传播分析问题，在一些场景上失去了结合其他语义信息用于优化的机会**

例子中转为虚表后，`obj`是函数参数，从常量传播分析的视角看，无法预知其虚表值是多少；而实际上我们可以根据Sub1无法再被进一步继承来判定其虚表内容

```
%14 = getelementptr inbounds ptr, ptr %13, i64 0
%15 = load ptr, ptr %14
%16 = call i32 %15(ptr %12)
```

> 二、将虚表及虚函数调用的具体实现封装在由runtime提供的intrinsic里。此时**LLVM无法感知intrinsic所包含的虚表及虚函数调用的语义信息**

仓颉 Cangjie

现代语言设计需求与传统编译架构之间存在Gap

Gap 3：语言独有特性的实现需求

源代码（rust）及预期效果：

```
fn foo() {
    let r;
    let cond: bool = …;
    let v = 10;
    if cond {
        let x = 5;
        r = &x;
    } else {
        r = &v;
    }

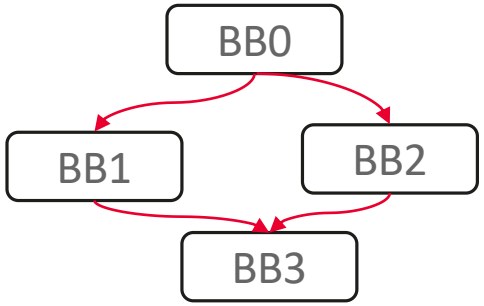
    // Error: r可能在引用一个生命周期已结束的变量
    println!("r: {r}");
}
```

要点：
- 数据流分析能力
- 生命周期分析能力（特别是因{}作用域导致的生命周期）
- 理解rust的所有权语义（例如borrow和move等）

- AST是一个树形的层级结构，非常贴近源码，难以进行数据流分析

- LLVM IR是一个铺平的结构（由BasicBlock构成的CFG），源码中 {} 导致的作用域层级信息已丢失

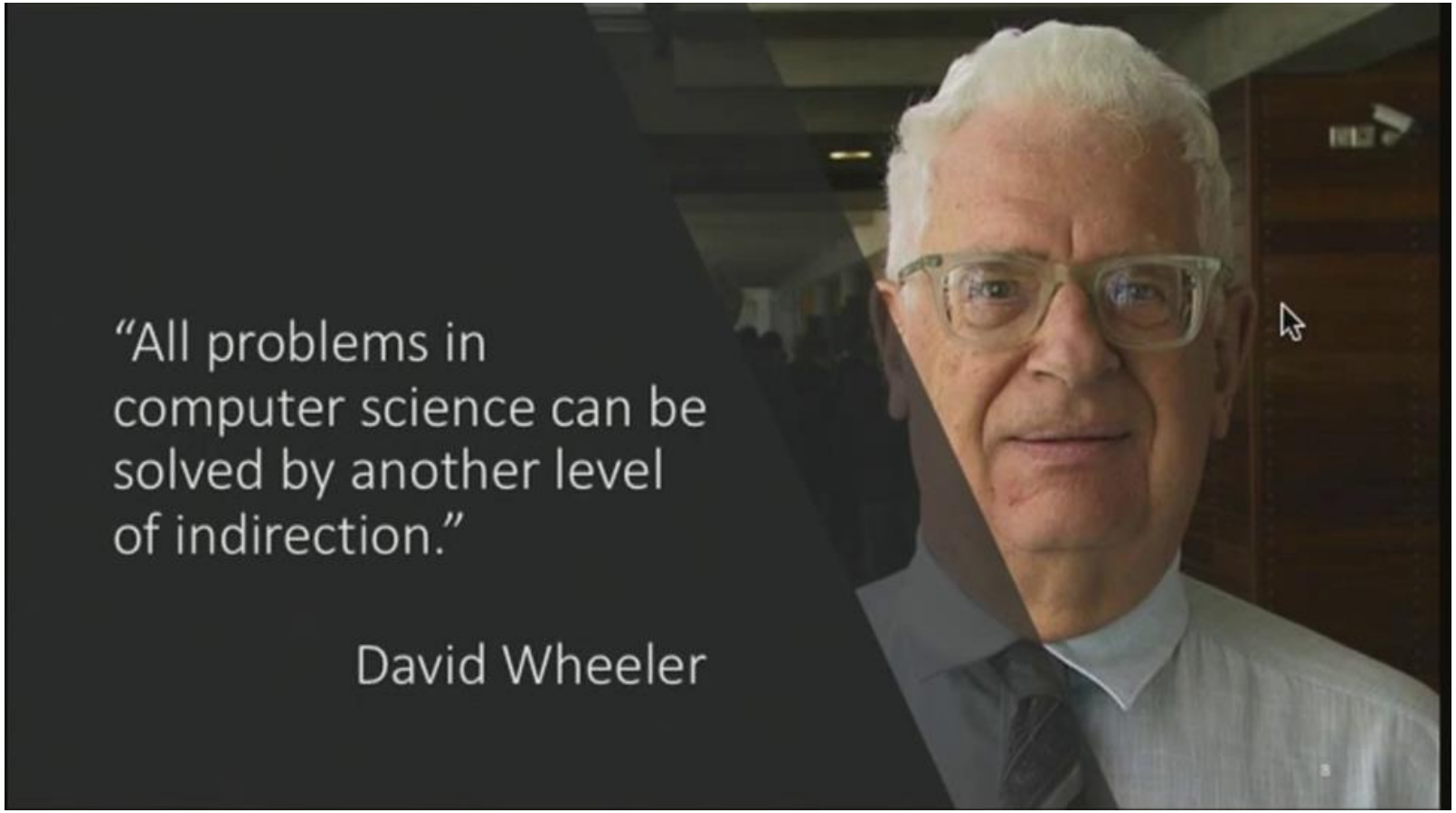
变量x在BB1中被定义，但铺平的结构无法重构源码中的{}导致的作用域范围



- LLVM IR不具有感知rust所有权语义能力

仓颉 Cangjie

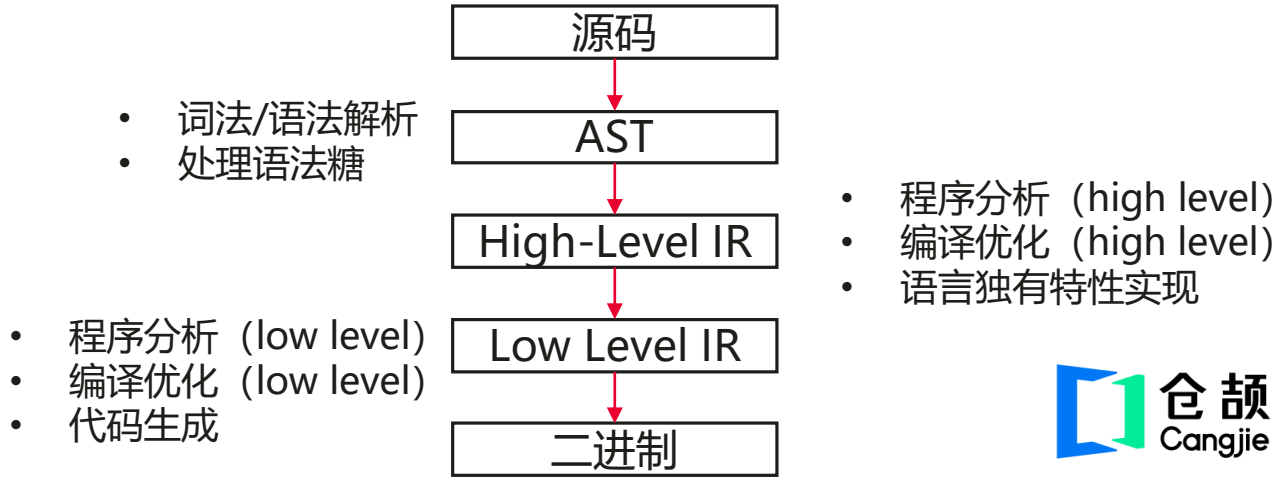
没有什么问题是加一个中间层解决不了的，如果有，就再加一层



High-Level IR：解决AST与Low-Level IR之间的Gap

- High-Level IR的设计思想和目标收益如下
  - > 抽象层次介于AST与Low-Level IR之间：
    - 相比Low-Level IR，保留更丰富的高层语义信息用于编译优化；更准确的源码位置映射关系用于告警报错
    - 相比AST，适当引入低层语义表达利于分析优化；更稳定（从语言演进的视角）的表达利于工程维护；
  - > 更显式的数据流/控制流信息利于程序分析和编译优化
  - > 提供语言独有特性的实现平台

- 凡事都会有代价，High-Level IR的代价则在于：
  - > 更复杂的编译前端
  - > 更多的编译开销（编译时间、编译资源占用）



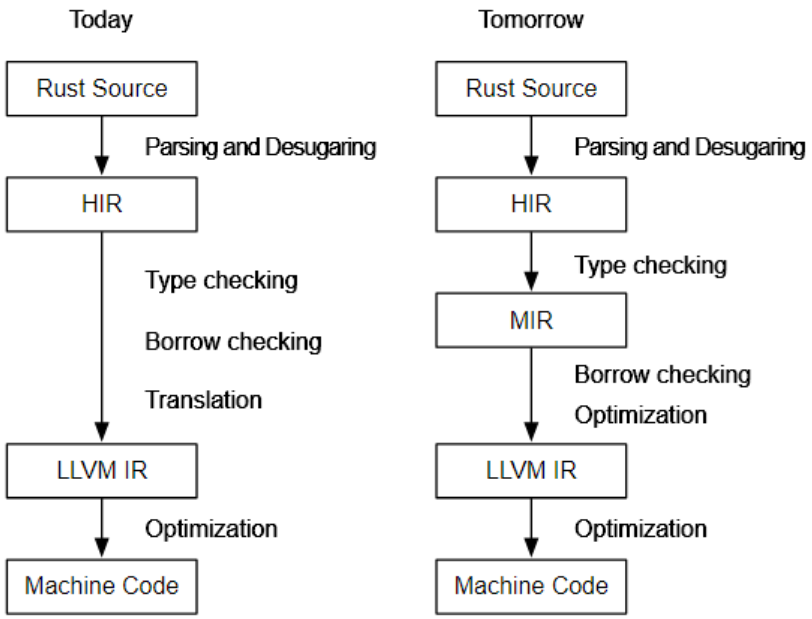
仓颉 Cangjie

目录

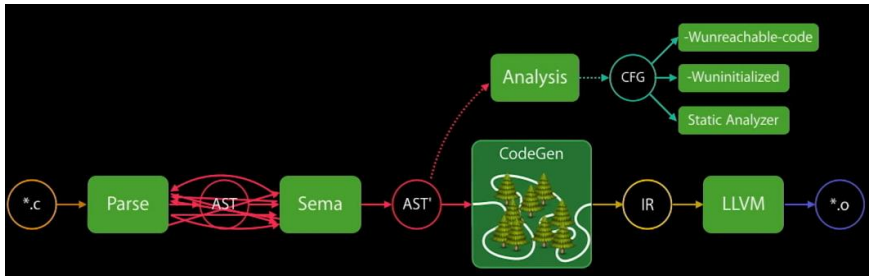
- 背景和动机
- 业界优秀实践简介
- Cangjie High-Level IR设计

High-Level IR in Rust & Swift：MIR

MIR in Rust Compiler



SIL in Swift Compiler





Introducing MIR | Rust Blog (rust-lang.org)

The MIR (Mid-level IR) - Rust Compiler Development Guide (rust-lang.org)

2015 LLVM Developers' Meeting: Joseph Groff & Chris Lattner "Swift's High-Level IR: A Case Study..." (youtube.com)

目录

- 背景和动机
- 业界优秀实践简介
- Cangjie High-Level IR设计

Cangjie High-Level IR (CHIR) 设计目标

- 解决AST与LLVM IR的语义Gap，相比LLVM IR保留**更多高层语义信息**用于程序分析和编译优化
  - > 仓颉源码中用户自定义类型包含的语义信息，如：类型继承关系，虚表信息等
  - > 仓颉表达式中包含的语义信息，如：循环结构中的循环变量、边界，代码块的作用域等

```
// CJ源码
class Base {
    open func foo() : Unit {...}
}

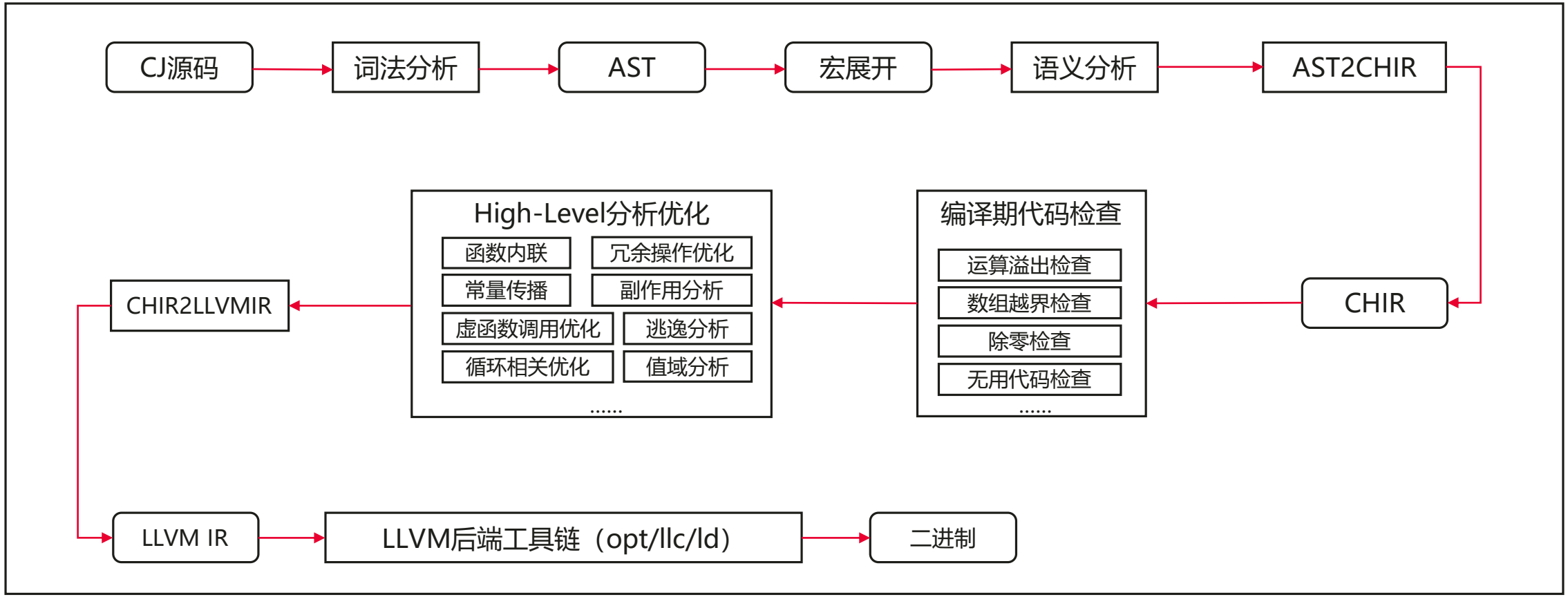
class SubClassA <: Base {
    override func foo() : Unit {...}
}
```

```
// CJ源码
var sum = 0
for (i in 0..100) {
   let innerVar = ...
   sum += i
}
```

- 相比AST，提供**数据流分析能力**使能程序分析和编译优化

- 通过CHIR来作为前端和后端之间的屏蔽解耦层，提供**隔离语法变化**和**后端实现无关**的通用分析优化平台

基于CHIR的仓颉编译流程概览



CHIR语法概览一

```
v ::=
    | l               // literal, including integer, float, char, bool and string etc
    | x               // variable, including global variable and local variable
    | f               // func identifier
    | im              // imported variable/func identifier
    | t               // type identifier
    | b               // block identifier
    | blockGroup

statement ::=
    | terminatorStmt  // statement which will trigger control-flow transition between blocks
    | trivialStmt                // other statements

// all statements inside will always be executed in order, last terminatorStmt will trigger control-flow
// transist to successor blocks or exit the belonging blockGroup
block ::=
    b: trivialStmt, ..., trivialStmt, terminatorStmt

// a control-flow graph built by all blocks and their control-flow transition dependencies inside. Always
// have a block as the entry of the control-flow graph
blockGroup ::=
    block, ..., block

func ::=
    'func' f(x1, ..., xn) : t
        blockGroup
```

CHIR示例一

源代码（CJ）：

```
func foo(x: Int32) {
    if (x == 10) {
        return x + 10
    }

    return x * 2
}
```

CHIR:

额外的属性信息用[]或//方式表示

```
Func @_ZN7default3fooEi([readOnly] %0: Int32) : (Int32) -> Int32 {  // loc: “helloCJ.cj"-1-1
[entry] Block #0:
  [ret] %1: Int32& = Allocate(Int32)
  GoTo(#1)
Block #1:
  %2: Int32 = Constant(10)
  %3: Bool = Equal(%0, %2)
  Branch(%3, #2, #3)
Block #2:
  %4: Int32 = Constant(10)
  %5: Int32 = Add(%0, %4)
  %6: Unit = Store(%5, %1)
  Exit
Block #3:
  GoTo(#4)
Block #4:
  %7: Int32 = Constant(2)
  %8: Int32 = Mul(%0, %7)
  %9: Unit = Store(%8, %2)
  Exit
}
```

Terminator语句触发控制流跳转

Block内语句顺序执行

设计要点一：High-Level语义支持（via BlockGroup嵌套）

```
terminatorStmt ::=
    | Exit
    | GoTo(b)
    | Branch(v, b1, b2)
    | ...

trivialStmt ::=
    | x: t = basicExpr
    | x: t = complexExpr

basicExpr ::=
    | UnaryOp(v)
    | BinaryOp(v1, v2)
    | Apply(f, v1, ..., vn)
    | Allocate(t)
    | Store(v1, v2)
    | Load(v)
    | Invoke(f, t, v1, ..., vn)
    | InstaceOf(v, t)
    | ...
```

```
complexExpr ::=
    | If(v, blockGroup1, blockGroup2)
    | Loop(v, blockGroup)
    | ForIn(v1, v2, blockGroup1, blockGroup2, blockGroup3)
    | Lambda(x1, ..., xn => blockGroup)
```

CHIR中引入了能表达High-Level语义的语句，可帮助保留更多语义信息：

- 例如Invoke、InstanceOf可以屏蔽底层虚表、对象内存布局实现细节，直接表示更贴近源码的高层语义信息
- 而原生的If、Loop、ForIn、Lambda语句**支持嵌套的BlockGroup**可以保留更多仓颉源码中天然存在的代码结构和语法糖信息。但特别需要注意的是，**支持嵌套BlockGroup的表达式不能破坏“Block中语句都是严格顺序执行”这一原则，否则将会使控制流分析变得更复杂**

CHIR示例二

源代码（CJ）：

```
func foo(x: Int32) {
    if (x == 10) {
        return x + 10
    }

    return x * 2
}
```

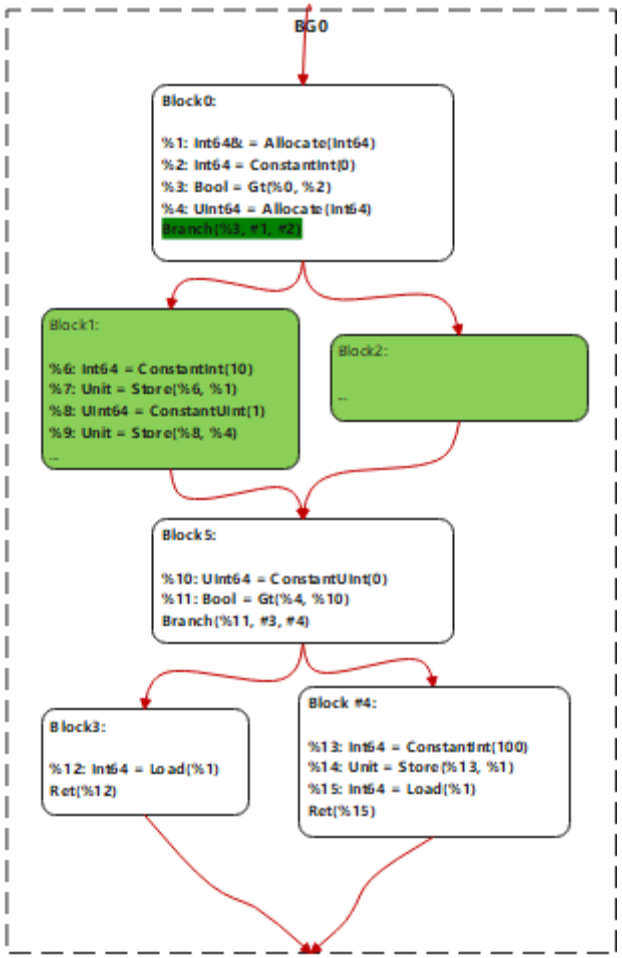
CHIR:

```
Func @_ZN7default3fooEi([readOnly] %0: Int32) : (Int32) -> Int32 {
[entry] Block #0:
  [ret] %1: Int32& = Allocate(Int32)
  GoTo(#1)
Block #1:
  %2: Int32 = Constant(10)
  %3: Bool = Equal(%0, %3)
  %4: Int32& = Allocate(Int32)
  If(%3, {
    Block #2:
      %5: Int32 = Constant(10)
      %6: Int32 = Add(%0, %5)
      %7: Unit = Store(%6, %1)
      %8: Int32 = Constant(1)
      %9: Unit = Store(%8, %4)
      Exit
  }, {
    Block #3:
      Exit
  })
  %10: Int32 = Load(%4)
  %11: Int32 = Constant(0)
  %12: Bool = Equal(%10, %11)
  Branch(%12, #5, #4)
Block #4:
  Exit
Block #5:
  %13: Int32 = Constant(2)
  %14: Int32 = Mul(%0, %13)
  %15: Unit = Store(%14, %1)
  Exit
}
```

特别注意，为了仍然能够保证Block #1的语句顺序执行，If的true分支中的return语句实际上是被delay到外面完成的

If语句中支持嵌套的BlockGroup，从而保留源码中If表达式的scope信息

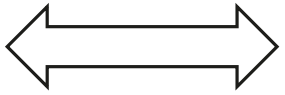
支持BlockGroup嵌套后变化和收益

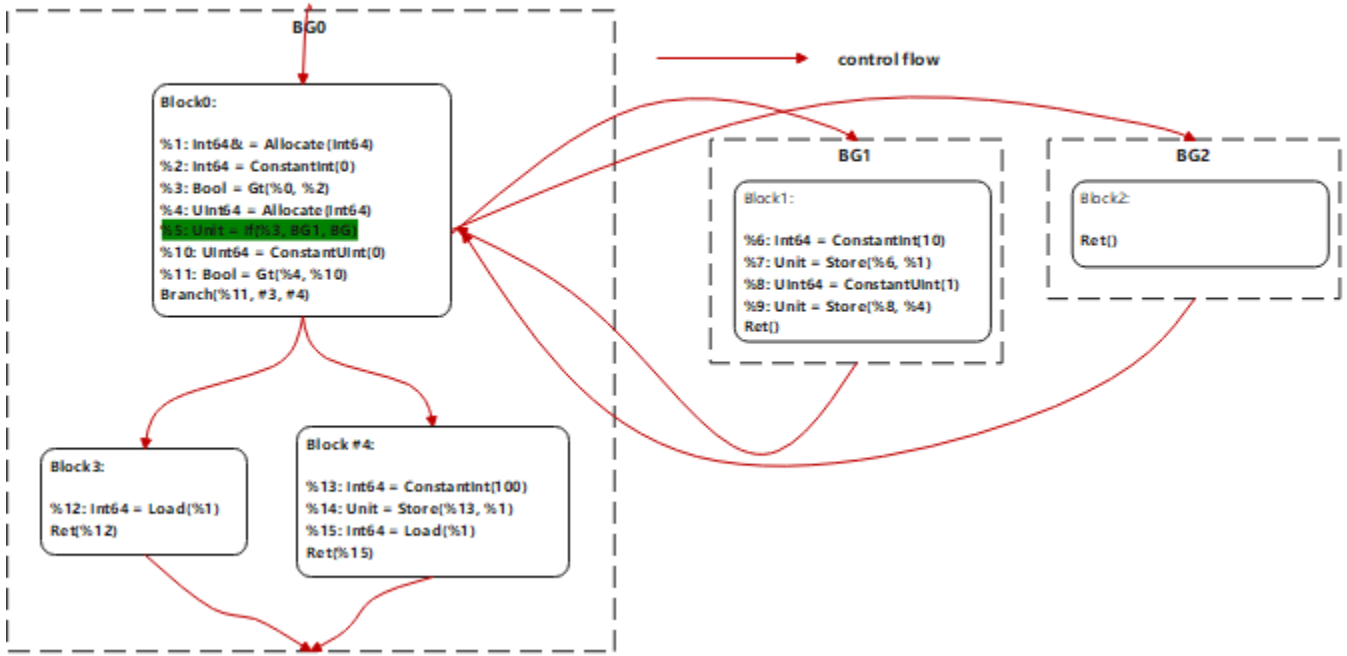


引入BlockGroup嵌套后：

- Block内语句一定能严格按顺序执行
- Block与Block之间的控制流关系构成CFG
- **部分语句执行过程中会进入到其嵌套的BlockGroup构成的sub-CFG中，但sub-CFG会保证将控制流返还给语句，然后继续顺序向下执行**

二者之间可以做等价变换





收益：

- BlockGroup本身可以承载源码中的scope语义信息，内层BG可以访问外层BG的值，反之不行
- 可基于嵌套的BlockGroup在IR中设计能表达更复杂语义的指令（如：for-in）

CHIR语法概览二

```
// declaration which defines a imported function or global variable from other packages
importDecl ::=
    ‘#from’ package_name 'import' im : t

// declaration which defines a user-customized type, including enum, struct and class
customTypeDecl ::=
    | enumDecl
    | structDecl
    | classDecl

// declaration which defines a global variable
globalVarDecl ::=
    v : t

// the basic compilation unit
Package ::=
    {importDecl}
    {customTypeDecl}
    {globalVarDecl}
    {func}
```

设计要点二：保留仓颉自定义类型语义信息

源代码（CJ）：

```
class Base {
    static let s : Int64 = 0
    let x : Int64
    var y : Int64

    init() {...}
    open func foo() : Unit {...}
    static func goo() : Unit {...}
    func koo() : Unit {...}
}

class SubClassA <: Base {
    override func foo() : Unit {...}
}
```

CHIR:

```
[static] [readOnly] @_ZN11default23Base1s : Int64&

class @_ZN7default4Base: Class-Base {
    [static] let @_ZN11default23Base1s : Int64
    let x : Int64
    var y : Int64
    func @_ZN11default23Baseinit : (Class-Base &) -> Unit
    func @_ZN11default23Basefoo : (Class-Base &) -> Unit
    [static] func @_ZN11default23Basegoo : () -> Unit
    func @_ZN11default23Basekoo : (Class-Base &) -> Unit
    vtable {
        foo : (Class-Base &)->Unit => @_ZN11default23Basefoo
    }
}
class @_ZN11default23SubClassA : Class-SubClassA <: @_ZN7default4Base {
    func @_ZN11default23SubClassAfoo : (SubClassA &) -> Unit
    vtable {
        foo : (Class-Base &)->Unit => @_ZN11default23SubClassAfoo
    }
}

func @_ZN11default23Baseinit : (Class-Base &) -> Unit {...}
func @_ZN11default23Basefoo : (Class-Base &) -> Unit {...}
[static] func @_ZN11default23Basegoo : () -> Unit {...}
...
```

保留类型成员变量布局信息+静态成员变量信息

自定义类型中的方法被翻译为全局函数的同时，保留其与自定义类型本身的关系

保留类型继承关系

原生支持vtable信息记录

仓颉 Cangjie

一些未来工作方向

- 结合指令和类型系统的设计，表达其他high-level语义以利于分析优化，如：box/unbox、
- 更好的支撑仓颉正在探索和设计的创新特性
- 作为未来仓颉开放生态中的一环，向生态开发者提供可扩展的程序分析、编译优化平台

WORKSHOP 研讨/分享/感悟

仓颉社区中的灵感碰撞 / 社区先行者的干货分享

仓颉 Cangjie

感谢您的观看!

关注“仓颉编程语言”公众号，
获取最新官方资讯!



“内联”前：

```
Func @_ZN7default3fooEi([readOnly] %0: Int32) : (Int32) -> Int32 {
[entry] Block #0:
  [ret] %1: Int32& = Allocate(Int32)
  GoTo(#1)
Block #1:
  %2: Int32 = Constant(10)
  %3: Bool = Equal(%0, %3)
  %4: Int32& = Allocate(Int32)
  If(%3, {
    Block #2:
      %5: Int32 = Constant(10)
      %6: Int32 = Add(%0, %5)
      %7: Unit = Store(%6, %1)
      %8: Int32 = Constant(1)
      %9: Unit = Store(%8, %4)
      Exit
  }, {
    Block #3:
      Exit
  })
  %10: Int32 = Load(%4)
  %11: Int32 = Constant(0)
  %12: Bool = Equal(%10, %11)
  Branch(%12, #5, #4)
Block #4:
  Exit
Block #5:
  %13: Int32 = Constant(2)
  %14: Int32 = Mul(%0, %13)
  %15: Unit = Store(%14, %1)
  Exit
}
```

If 语句转换为 Branch 指令和“内联”后的两个 Block。Block内的Exit指令替换为到If语句后代码的GoTo语句

“内联”后：

```
Func @_ZN7default3fooEi([readOnly] %0: Int32) : (Int32) -> Int32 {
[entry] Block #0:
  [ret] %1: Int32& = Allocate(Int32)
  GoTo(#1)
Block #1:
  %2: Int32 = Constant(10)
  %3: Bool = Equal(%0, %3)
  %4: Int32& = Allocate(Int32)
  Branch(%3, #2, #3)
Block #2:
  %5: Int32 = Constant(10)
  %6: Int32 = Add(%0, %5)
  %7: Unit = Store(%6, %1)
  %8: Int32 = Constant(1)
  %9: Unit = Store(%8, %4)
  GoTo(#6)
Block #3:
  GoTo(#6)
Block #6:
  %10: Int32 = Load(%4)
  %11: Int32 = Constant(0)
  %12: Bool = Equal(%10, %11)
  Branch(%12, #5, #4)
Block #4:
  Exit
Block #5:
  %13: Int32 = Constant(2)
  %14: Int32 = Mul(%0, %13)
  %15: Unit = Store(%14, %1)
  Exit
}
```

若进一步将Block #6复制到#2和#3内，则可进一步通过常量传播消除额外引入的变量%4的读写和判定操作